

## Лабораторна робота № 7.

**Тема:** Динамічні списки даних у Python

**Мета:** Засвоїти базові поняття та набуті практичних навичок роботи з динамічними списками, зрізами та базовими алгоритмами обробки числових послідовностей у мові програмування Python.

### КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

#### 1. Створення та обробка списків

**Список (list)** – це впорядкована змінна структура даних.

- **Зчитування списку цілих чисел з одного рядка:**

Python

```
numbers = list(map(int, input().split()))
```

- **Перетворення елементів у рядок та виведення:**

Python

```
print(*numbers) # виведення через пропуск
```

```
a=[1,2,3,4]
```

```
print(*a)
```

У терміналі: 1 2 3 4

#### 2. Зрізи (Slices)

Зрізи дозволяють отримувати підсписки за формою `list[start:stop:step]:`

- `lst[len(lst)//2:]` – друга половина списку.
- `lst[1::2]` – елементи з непарними індексами.
- `lst[::-1]` – список у зворотному порядку.

#### 3. Базові методи та операції

- **Додавання та видалення:** `append()`, `pop()`, `remove()`.
- **Пошук та агрегація:** `min()`, `max()`, `len()`, `sum()`, `count()`, `index()`.

#### Приклад:

Напишіть програму, яка роздруковує лише ті елементи введеного списку унікальних значень, що менше введеного значення `n`.

#### Вхідні дані:

```
2 7 11 3 8 90 144 15 5
```

```
94
```

## Вихідні дані:

2 7 11 3 8 90 15 5

## Аналіз умови та вхідних даних

### Вхідні дані:

1. Перший рядок містить послідовність чисел, розділених пропуском.
2. Другий рядок містить одне ціле число **n**, з яким порівнюються елементи списку.

**Вихідні дані:** Рядок із числами (із початкового списку), які за значенням строго менші за **n**. Порядок виведення елементів має відповідати їхньому початковому порядку у списку.

## Ключові етапи алгоритму

### 1. Зчитування та перетворення вхідних даних:

- Зчитати перший рядок за допомогою `input()`.
- Розбити рядок на окремі підрядки за допомогою методу `.split()`.
- Перетворити кожен елемент на ціле число (`int`) та сформувати список `numbers`.
- Зчитати другий рядок і перетворити його на ціле число `n = int(input())`.

### 2. Фільтрація елементів:

- Пройтися по кожному елементу списку `numbers`.
- Для кожного елемента перевірити умову: `item < n`.
- Якщо умова виконується – зберегти елемент у новий список або одразу вивести.

### 3. Формування та виведення результату:

- Вивести відфільтровані елементи в один рядок через пропуск.

## Приклади програмної реалізації

### Варіант 1: Класичний прохід циклом `for`

Python

```
# 1. Зчитування даних
numbers = list(map(int, input().split()))
n = int(input())
result = []
# 2. Фільтрація
for num in numbers:
    if num < n:
        result.append(num)
```

```
# 3. Виведення (оператор распакування * виводить елементи  
через пропуск)  
print(*result)
```

## Варіант 2: Використання спискового включення (List Comprehension — Pythonic way)

Python

```
numbers = list(map(int, input().split()))  
n = int(input())  
# Створення нового списку за допомогою фільтрації у List  
Comprehension  
result = [num for num in numbers if num < n]  
print(*result)
```

## ЗАВДАННЯ ДЛЯ САМОСТІЙНОЇ РОБОТИ

### РІВЕНЬ 1: БАЗОВИЙ

#### Завдання 1. Друга половина списку

Створіть список на основі введеної послідовності цілих чисел і надрукуйте другу половину списку.

**Вхідні дані:** 7 2 1 0 4 2 5

**Вихідні дані:** 0 4 2 5

#### Завдання 2. Елементи, більші за n

Напишіть програму, яка роздруковує лише ті елементи введеного списку унікальних значень, що менше введеного значення n.

**Вхідні дані:**

2 7 11 3 8 90 144 15 5

94

**Вихідні дані:** 2 7 11 3 8 90 15 5

#### Завдання 3. Кількість додатних елементів

Знайдіть кількість додатних елементів у введеному списку чисел.

**Вхідні дані:** 2 -4 5 6 -3

**Вихідні дані:** 3

#### Завдання 4. Елементи з непарними індексами

Виведіть всі елементи списку з непарними індексами.

**Вхідні дані:** 1, 2, 3, 4, 5, 6, 7, 8, 9

**Вихідні дані:** 1, 3, 5, 7, 9

#### **Завдання 5. Унікальні елементи**

Виведіть ті елементи списку, які зустрічаються в ньому лише один раз, у початковому порядку.

**Вхідні дані:** 4 5 6 6 6 5 4 4 7 4

**Вихідні дані:** 7

#### **Завдання 6. Елементи, більші за попередній**

Надрукуйте всі елементи, які перевищують попередній елемент списку.

**Вхідні дані:** 5 8 0 2 9 4 1

**Вихідні дані:** 8 2 9

#### **Завдання 7. Елементи, більші за двох сусідів**

Знайдіть кількість елементів, які за значенням більші за обох своїх сусідів (перший та останній елементи не враховуються).

**Вхідні дані:** 1 3 2 4 2

**Вихідні дані:** 2

#### **Завдання 8. Статистика числового списку**

Отримайте послідовність цілих чисел і виведіть: найменше число, найбільше число, кількість чисел та їх середнє значення.

**Вхідні дані:** 1 3 7 5

**Вихідні дані:**

1

7

4

4.0

#### **Завдання 9. Фільтрація додатних чисел**

Створіть програму для друку списку лише із введених додатних чисел у вигляді об'єкта `list`.

**Вхідні дані:** 0 9 -4 6 8 -15 4

**Вихідні дані:** [9, 6, 8, 4]

#### **Завдання 10. Перетворення списку чисел у єдине ціле число**

Напишіть програму для перетворення списку декількох цілих чисел у єдине ціле число.

**Вхідні дані:** 1 7 9 4

**Вихідні дані:** 1794

## РІВЕНЬ 2: ОСНОВНИЙ / СЕРЕДНІЙ

### Завдання 11. Циклічний зсув вправо

Циклічно посуньте елементи списку вправо на одну позицію.

**Вхідні дані:** 2 0 2 5

**Вихідні дані:** 5 2 0 2

### Завдання 12. Обмін мінімуму та максимуму

У послідовності унікальних цілих чисел поміняйте місцями мінімальний та максимальний елементи.

**Вхідні дані:** 1 9 12 5 3 8

**Вихідні дані:** 12 9 1 5 3 8

### Завдання 13. Підрахунок рівних пар

Підрахуйте, скільки пар елементів мають однакові значення (кожна пара враховується один раз).

**Вхідні дані:** 1 2 2 2 3 3 4

**Вихідні дані:** 4

### Завдання 14. Перестановка сусідніх елементів

Обміняйте сусідні елементи у парах ( $A[0]$  з  $A[1]$ ,  $A[2]$  з  $A[3]$  і т.д.). Непарний останній елемент залишається на місці.

**Вхідні дані:** 1 4 5 3 7

**Вихідні дані:** 4 1 3 5 7

### Завдання 15. Медіана трьох чисел

Знайдіть медіану з трьох введених цілих чисел.

**Вхідні дані:** 25 50 11

**Вихідні дані:** 25

### Завдання 16. Шеренга учнів

Дано спадаючу послідовність зросту учнів та зріст нового учня. Визначте номер місця для нового учня (якщо зріст однаковий, він стає після них).

**Вхідні дані:**

165 163 160 160 157 157 155 154

160

**Вихідні дані:** 5

### Завдання 17. Перетворення з двійкової системи у десяткову

Напишіть програму для перетворення двійкового числа у десяткове.

**Вхідні дані:** 101010111

**Вихідні дані:** 343

### Завдання 18. Покрокове видалення кожного третього елемента

Видаляйте кожний третій елемент із цілочисельного списку і друкуйте результуючий список, доки він не стане порожнім.

**Вхідні дані:** 2 5 8 9 4 78 7 1

**Вихідні дані:**

[2, 5, 9, 4, 78, 7, 1]

[2, 5, 4, 78, 7, 1]

...

[]

## РІВЕНЬ 3: ПОГЛИБЛЕНИЙ / ВИСОКИЙ

### Завдання 19. Задача про автобуси

У трьох автобусах розсадили дітей так, що їхня кількість у різних автобусах різна. Знайдіть найменшу кількість дітей, яку потрібно пересадити, щоб у кожному автобусі їх стало порівно. Якщо це зробити неможливо, виведіть NO SOLUTIONS.

**Вхідні дані:** 2 5 8

**Вихідні дані:** 3

### Завдання 20. Пошук позицій числа у списку

Прийміть на вхід список чисел і окреме число, після чого виведіть усі позиції (нумерація з 1), на яких це число зустрічається в списку. Якщо число не знайдено, виведіть None.

**Вхідні дані:**

5 7 3 4 9 8 4 7 4

4

- **Вихідні дані:** 4 7 9

### Завдання 21. Стиснення списку (зсув нулів управо)

Перемістіть усі ненульові елементи в ліву частину списку без зміни їхнього порядку, а всі нулі – в праву. Завдання потрібно виконати за один прохід по списку та без використання додаткового списку.

**Вхідні дані:** 6 0 3 0 5 0 0 4

**Вихідні дані:** 6 3 5 4 0 0 0 0